

Praktikum 3: Quick Sort und Heap Sort

1. Problematisch für die Laufzeit des Quick-Sort ist eine ungünstige Wahl des Pivot-Elements. Im klassischen Quick-Sort wird stets das Element am rechten Rand als Pivot-Element ausgewählt. Implementieren Sie eine Variante des Quick-Sort, bei der das Element am linken Rand, das Element am rechten Rand, und das Element in der Mitte des Arrayabschnitts betrachtet werden und das wertmäßig mittlere Element dieser drei als Pivot-Element verwendet wird

Führen Sie diesen modifizierten Algorithmus mit den vorgegebenen Folgen aus und vergleichen Sie die Anzahl der Vergleiche.

Kann durch die Modifikation ein besseres Laufzeitverhalten als $O(n \log n)$ erreicht werden?

2. Mit Hilfe eines Heap lässt sich auch effizient eine Vorrang-Warteschlange implementieren. Dabei sollte das Element mit der höchsten Priorität immer an der Wurzel des Heap stehen.

Erstellen Sie eine Vorrang-Warteschlange-Klasse, die mit einem Heap arbeitet. Diese Klasse sollte mindestens folgende Methoden implementieren:

- `insert(item, priority)`: Fügt ein Element mit einer bestimmten Priorität in die Warteschlange ein.
- `pop()`: Entfernt das Element mit der höchsten Priorität und gibt es zurück.
- `peek()`: Gibt das Element mit der höchsten Priorität zurück, ohne es zu entfernen.
- `is_empty()`: Überprüft, ob die Warteschlange leer ist.

Musterlösung Praktikum 3: Quick Sort und Heap Sort

1 Quick-Sort mit modifizierter Pivot-Wahl

1.1 Idee der Modifikation

Beim klassischen Quick-Sort wird das Pivot-Element stets als rechtes Array-Element gewählt. Dies kann im ungünstigsten Fall (z.,B. bereits sortierte Eingaben) zu einer sehr schlechten Laufzeit führen. Die modifizierte Variante betrachtet:

- linkes Element
- rechtes Element
- mittleres Element

und wählt das **Median-Element dieser drei Werte** als Pivot. Diese Strategie wird als *Median-of-Three* bezeichnet.

1.2 Auswirkungen auf die Anzahl der Vergleiche

Die zusätzliche Bestimmung des Median-Elements benötigt eine konstante Anzahl an Vergleichen:

Bei der Median-of-Three-Pivotwahl müssen zunächst die drei Kandidaten (linkes, mittleres, rechtes Element) sortiert werden, um den Median zu bestimmen. Dies erfordert im Worst Case **3 Vergleiche** pro Partitionierungsschritt. Im Vergleich dazu benötigt der klassische Quick-Sort mit fester Pivotwahl (rechtes Element) **keine zusätzlichen Vergleiche** zur Pivotbestimmung, kann aber in der Partitionierung sehr unausgeglichene Teile erzeugen. Die 3 zusätzlichen Vergleiche pro Ebene sind ein vernachlässigbarer Overhead im Vergleich zur Gesamtzahl der Vergleiche, führen aber zu:

- ausgeglicheneren Partitionen
- weniger Rekursionstiefe im Durchschnitt
- geringerer Wahrscheinlichkeit für den Worst Case

1.3 Vergleich mit klassischem Quick-Sort

1.3.1 Worst Case beim klassischen Quick-Sort (Pivot = rechts)

Der Worst Case tritt auf bei:

- bereits sortierten Arrays
- umgekehrt sortierten Arrays

In diesen Fällen erzeugt die Partitionierung immer einen Teilbereich der Größe $n - 1$ und einen der Größe 0.

Die Rekurrenz lautet:

$$T(n) = T(n - 1) + \Theta(n)$$

Durch wiederholtes Einsetzen ergibt sich:

$$T(n) = T(n - 1) + cn = T(n - 2) + (n - 1)c + cn = \dots = \sum_{k=1}^n ck = c \cdot \frac{n(n + 1)}{2} = \Theta(n^2)$$

1.3.2 Worst Case beim Median-of-Three Quick-Sort

Durch die Auswahl eines „zentraleren“ Pivot-Elements wird die Wahrscheinlichkeit für stark unausgeglichene Partitionen reduziert.

Wichtig: Der Worst Case bleibt theoretisch weiterhin $\Theta(n^2)$. Dieser tritt jedoch deutlich seltener auf, da es extrem unwahrscheinlich ist, dass bei drei zufällig gewählten Positionen stets das kleinste oder größte Element als Median ausgewählt wird.

Wann tritt der Worst Case noch auf? Der Worst Case kann eintreten, wenn die Eingabe so konstruiert ist, dass bei jeder Partitionierung trotz Median-of-Three eine extrem unausgeglichene Zerlegung entsteht (z. B. Größe $n - 2$ und 1). Dies ist möglich bei:

- **Speziell manipulierten Daten:** Wenn ein Angreifer oder ein Algorithmus das Array so anordnet, dass von den drei betrachteten Positionen immer zwei die kleinsten bzw. größten Elemente enthalten, wird das verbleibende Element zum Pivot und führt ggf. zu einer stark unausgegliehenen Partitionierung.
- **Permutationen mit periodischem Muster:** Es existieren konkrete Permutationen, bei denen Median-of-Three nicht verhindern kann, dass oft extreme Pivot-Wahlen getroffen werden. Diese sind jedoch mathematisch konstruiert.

In der Praxis treten solche Fälle jedoch so selten auf, dass Median-of-Three als robuste Heuristik gilt.

1.4 Fazit

- Keine Verbesserung der asymptotischen Worst-Case-Komplexität
- Robust gegenüber vorsortierten Eingaben

2 Vorrang-Warteschlange mit Heap

2.1 Datenstruktur

Die Vorrang-Warteschlange wird als **Max-Heap** implementiert:

- vollständiger Binärbaum
- Elternknoten $\geq$ Kinderknoten

Zusammenhang mit Heap-Sort aus der Vorlesung Die hier verwendete Heap-Datenstruktur entspricht der im Rahmen von Heap-Sort eingeführten Struktur. Der zugrunde liegende vollständige Binärbaum lässt sich effizient in einem Array speichern.

Terminologie-Hinweis: In der Vorlesung heißt die Abwärts-Operation `MAX-HEAPIFY` – sie stellt die Heap-Eigenschaft für einen Teilbaum wieder her. `sift_down` ist funktional identisch damit, folgt aber der in der übrigen Literatur gebräuchlicheren Richtungsbezeichnung (`sift` = „sieben/sinken lassen“).

`sift_up` hat kein direktes Gegenstück in der Vorlesung, weil Heap-Sort nie ein Element einfügt: Dort wird nur nach unten geheapifiziert. Für eine vollständige Priority Queue mit `insert()` wird jedoch auch die Aufwärts-Richtung benötigt.

2.2 Operationen

2.2.1 `insert(item, priority)`

- Einfügen am Ende des Heaps
- Wiederherstellung der Heap-Eigenschaft durch `sift_up`
- Laufzeit: $\mathcal{O}(\log n)$

2.2.2 `pop()`

- Entfernen der Wurzel (höchste Priorität)
- Ersetzen durch letztes Element
- Wiederherstellung durch `sift_down` bzw. `MAX-HEAPIFY`
- Laufzeit: $\mathcal{O}(\log n)$

2.2.3 `peek()`

- Zugriff auf die Wurzel
- Laufzeit: $\mathcal{O}(1)$

2.2.4 `is_empty()`

- Prüfen, ob der Heap leer ist
- Laufzeit: $\mathcal{O}(1)$

2.3 Korrektheit

Die Heap-Eigenschaft garantiert:

Maximale Priorität befindet sich stets an der Wurzel

Damit erfüllt die Datenstruktur die Anforderungen einer Vorrang-Warteschlange.

2.4 Komplexitätsübersicht

Operation	Laufzeit
<code>insert</code>	$\mathcal{O}(\log n)$
<code>pop</code>	$\mathcal{O}(\log n)$
<code>peek</code>	$\mathcal{O}(1)$
<code>is_empty</code>	$\mathcal{O}(1)$